

AstraNotes

Week 8.1 — Collaborative Git Workflow

Jonathan Fong · CSEN 296B · May 25, 2026

Part 1 — Collaboration Log

This lab was completed as an individual simulation. Claude Code was used to execute the full Git workflow autonomously — branch creation, code changes, build verification, and push — while I played both the author and reviewer roles on GitHub.

Role	Action	Branch	Commit
Author	Created branch, implemented case-insensitive searchByTitle, added 2 GTest cases, verified 5/5 tests, pushed	feature/search-improvement	541e4b2
Author	Created branch, replaced std::map with unordered_map in NoteManager, verified 3/3 tests, pushed	fix/test-cleanup	4193811
Reviewer	Left review comment on toLower performance and empty-query coverage gap on PR #1	feature/search-improvement	—
Reviewer	Left review comment on unordered_map iteration-order behavior change on PR #2	fix/test-cleanup	—
Author	Merged PR #2 (fix/test-cleanup) — low risk, fully traceable, no open review questions	main	—
Author	Left PR #1 open — empty-query test case missing per Definition of Done testability gate	feature/search-improvement	—

Part 2 — Branch and PR Workflow Summary

Branch 1: feature/search-improvement

Purpose: Implement FR-6 (refined) — case-insensitive title substring search in NoteManager.searchByTitle. The original implementation from Week 6 was a stub that returned an empty vector. This branch replaced it with a real implementation satisfying the FR-6 requirement that search be case-insensitive and produce no file I/O.

Files changed:

- src/controller/NoteManager.cpp — replaced stub with case-insensitive substring search using std::transform + std::tolower via a toLower lambda; added #include <algorithm> and #include <cctype>

- test/test_note_creation.cpp — added SearchByTitle.CaseInsensitive_UpperQueryMatchesLowerTitle and SearchByTitle.NoMatch_ReturnsEmptyVector test cases

Commit message:

```
feat: make searchByTitle case-insensitive per FR-6
```

Test result: 5/5 passed (ctest exit 0)

Branch 2: fix/test-cleanup

Purpose: Resolve the std::map vs unordered_map mismatch flagged in the Week 7.2 testing log. NFR-1 requires O(1) average UUID lookup via an unordered structure. The Week 6 implementation used std::map, which gives O(log n). This branch corrects that gap with a minimal, targeted change.

Files changed:

- src/controller/NoteManager.h — replaced #include <map> with #include <unordered_map>; updated notes_ field type and getNotes() return type; added NFR-1 traceability comment
- src/controller/NoteManager.cpp — updated getNotes() return type to match header

Commit message:

```
fix: replace std::map with unordered_map in NoteManager per NFR-1
```

Test result: 3/3 passed (ctest exit 0)

Workflow Sequence

Step	Action
1	Confirmed main was clean (one untracked TEST.txt, intentionally ignored)
2	Created feature/search-improvement from main
3	Implemented searchByTitle + 2 GTest cases
4	Encountered stale Xcode SDK build cache — deleted build/ and reconfigured clean
5	Verified 5/5 tests pass; committed and pushed branch
6	Returned to main; created fix/test-cleanup
7	Swapped std::map to unordered_map; verified grep showed no CLIView map-specific usage
8	Verified 3/3 tests pass; committed and pushed branch
9	Opened PR #1 and PR #2 on GitHub; added descriptions and review comments

10	Merged PR #2; left PR #1 open pending empty-query test case
----	---

Part 3 — PR Summary

PR #1: feature/search-improvement

Title: feat: make searchByTitle case-insensitive per FR-6

This PR replaces the searchByTitle stub in NoteManager.cpp with a real case-insensitive substring search implementation, satisfying FR-6 from the refined requirement baseline: "Search shall be case-insensitive. No file I/O shall occur during search."

The implementation uses a toLower lambda built on std::transform and std::tolower, applied to both the query and each note title before calling std::string::find. Two GTest cases were added: one confirming that an uppercase query matches a lowercase title, and one confirming that a non-matching query returns an empty vector with no exception.

Reviewer focus areas:

- The toLower lambda copies strings by value — confirm this is acceptable at expected collection sizes (up to 10,000 notes per NFR-1)
- Empty-query behavior: std::string::find("") returns 0 for any string, meaning every note matches — verify this aligns with FR-6's "empty query returns the full collection" requirement and add an explicit test case
- Return type is std::vector<std::string> (titles only) — confirm this is sufficient for the CLI use case or whether UUIDs need to accompany results

Part 4 — Review Feedback Example

Comment on PR #1 — searchByTitle implementation (NoteManager.cpp)

Posted on the toLower lambda and loop body in the diff:

The toLower lambda copies s by value on every call, so each note title is copied twice per search — once for the query conversion and once per note in the loop. For a 10,000-note collection (NFR-1 scale) this could add up. One option is to convert each title inline rather than through a second lambda call. Not blocking at current sprint scale — but worth logging as a known cost before Sprint 3 when search is the primary feature. Also: what does this return when query is ""? std::string::find("") on any string returns 0, so every note matches. That is actually correct per FR-6 ("empty query returns the full collection") but it is not tested explicitly. Suggest adding a D4-equivalent test case before merge.

Comment on PR #2 — notes_ field declaration (NoteManager.h)

Posted on the unordered_map notes_ field in the diff:

The switch from `std::map` to `std::unordered_map` is correct for NFR-1, but it changes the iteration order of `getNotes()`. `std::map` iterates in sorted key order; `unordered_map` has no guaranteed order. `CLIVView` currently calls `getNotes()` to render the note list — users may now see notes listed in a different order each session. This is not a bug given current requirements, but it should be documented in the README or a planning note before Sprint 3 adds more list-facing features. Recommend merging as-is and opening a follow-up backlog item to decide whether display order needs to be stable.

Part 5 — Merge and Merge-Readiness Note

PR #2 — Merged

`fix/test-cleanup` was merged into `main`. The change is two files and four lines, with a clear NFR-1 citation, no new dependencies, and a verified `grep` confirming no `CLIVView` map-specific API usage that would break under `unordered_map`. All three tests pass. The Definition of Done's traceability gate is satisfied by the inline NFR-1 comment added to the `notes_` field declaration. No open review questions remained after the iteration-order comment was posted and acknowledged.

PR #1 — Not Yet Ready

`feature/search-improvement` was not merged. The implementation is correct and all five tests pass, but the Definition of Done's testability gate requires that every explicitly stated requirement behavior be covered by a test. FR-6 states "an empty query shall return the full collection" — this behavior is not yet tested. The review comment identified this gap. The PR will be ready to merge after a D4-equivalent test case (empty query returns all notes, no exception) is added and `cctest` confirms 6/6.

Part 6 — Refactor Note

The `fix/test-cleanup` PR surfaced a behavioral side effect worth tracking: switching from `std::map` to `std::unordered_map` in `NoteManager` removes the implicit sorted iteration order that `std::map` provided. `CLIVView`'s `renderNoteList` iterates `getNotes()` directly, so the rendered note list is now in hash-table order — non-deterministic and not necessarily consistent between sessions.

This is not a defect today because no requirement specifies display order. But as Sprint 3 adds more list-facing features (search results, filtered views), a silent non-deterministic order will become harder to explain to a user. The right resolution is to decide explicitly: either sort by creation timestamp in `renderNoteList` before display, or add a `display-order` field to the `Note` model. Either option is a one-sprint task. The refactor note is: add a backlog item before Sprint 3 to define and enforce a stable display order, and ensure it is a documented design decision rather than an accident of whichever container the controller happens to use.

Part 7 — AI Use Reflection

Claude Code handled the full workflow autonomously — branch creation, code changes, build verification, and push — without manual intervention. The `searchByTitle` implementation it produced was correct on the first pass. The `unordered_map` swap was equally clean: two files, four lines, no logic changes, and a proactive `grep` to verify `CLView` had no map-specific API usage before committing.

One real issue arose during Branch 1: the build failed with a stale Xcode SDK path in the cached build directory. Claude Code diagnosed the cause correctly, deleted the build directory, reconfigured from scratch, and continued without prompting me. That was the kind of autonomous recovery the assignment is asking for.